Posts

emsenn

June 9, 2019

Contents

Introduction

Posts

Notes on "Guix Reference Manual," Sections 1-3

I've just installed the *Guix System Distribution* on my desktop, and in an attempt to learn it fully I am going to read the manual and record notes here.

The full text of the documentation is available at `https://www.gnu.org/software/guix/manual/en/guix.html`

The manual says *GNU Guix* is a package management tool, letting unprivileged users manage packages. It can be installed on an existing GNU/Linux system, or as a standalone operating system, *Guix System*. That's what I'm doing.

Guix implements the "'functional package management' discipline," apparently pioneered by *Nix*, another Linux distribution.

The package build/install process is a *fuction*. Its inputs are the build script, the compiler, dependent libraries, and it returns an installed package.

Packages can only refer to packages and such that were passed as inputs, and can't modify a system outside of its build and installation directories.

The results of package build functions is cached in "the Store," under a folder that has a hash of all the inputs, so it's appropriately unique. That's pretty clever! It's, by default, under `/gnu/store/`.

So, on to the *Guix System*. It comes with some core packages...

You "declare all aspects of the operating system configuration and *Guix* takes care of instantiating the configuration in a transactional, reproducible, and stateless fashion."

What... does that mean? Is it like *Emacs*, where you write an initialization file and that gets loaded on startup? And then... it checks the hash that the inputs of your build function calls against the packages that are there to see if things need to be installed?

Maybe I'm wrong, but I think that might be it. So, I've installed a few packages since I started up the system... I wonder where my system init is.

(Also, isn't that how *Arch* used to work, back when? A series of startup scripts that you could just... edit? I miss that; I had a very speedy netbook because of it.)

The next section of the manual is for how to install *Guix* ontop of another Linux, so I'll skip it for now; hopefuly that's fine.

The next section I already read before installing the system, because it's how to install the *Guix System*.

Well... I didn't read it, as such. I skimmed it. Let me read it thoroughly now.

So, it starts off with some limitations - that's a good idea. I should start off my project installation guides with disclosing limitations/failures of the project. *Logical Volume Management* is missing, but I don't know what that is and so I don't think I'll miss it. KDE is also missing. That's fine, it's never been my preferred window manager. I'm using MATE right now; I made it all gray and blue to look like old-school Windows.

It also warns me that wifi support is ify. My wifi chip does work, but I'm also wired into the router so that's irrelevant to me.

Okay, blah blah, more installation instructions. Nothing that seems too important: I think I'm ready to start using the system.

Notes on Installing Writefreely on a Guix System

Before lunch I successfully got *Nginx* to run on my desktop computer, which is running the *GuixSD* operating system.

My goal is still getting a federated communication platform up, though the more I give consideration to it, the more I think this might be the opportunity and excuse I need to stop microblogging: maybe email should be my federated communication platform of choice. Or, letter-writing, which I think is probably still best implemented through email.

That said, I would like to host my documents through a *WriteFreely* instance, so I can be followed via *ActivityPub*, so I'm going to get on with

that, rather than shift gears from *Nginx* to email.

So... I hate to say it but, I want to do this right, and I think that means creating a *Guix* package for Writefreely.

Dang. Back to the *Guix Reference Manual*, this time Section 14: "Contributing," since it has a subsection on packaging.

This section starts with some instructions for setting up the "Perfect Setup," which looks like it'd be really useful... if I knew more about Emacs and such.

So I'm going to skip to 14.4, "Packaging Guidelines."

And... realize I'm way in over my head. Happenstance, I've just received a message from @roptat@framapiaf.org about my progress with learning *Guix*, so I'm going to ask them if making a package is the right way to go, before I sink my teeth into this. They've answered:

> I don't know this software, so I can't be sure. I think you will need to create a package for it first, and then a service. They are two separate things. The service-type simply uses the package defined in guix.
>
> For writting your first package, I suggest reading `https://www.gnu.org/software/guix/blog/2018/a-packaging-tutorial-for-guix/`
>
> Then you'll probably have to look at actual package definitions for go packages that are a bit different from gnu packages. See go-build-system in the manual

I asked them to explain what they meant by "[t]he service-type simply uses the package defined in *[G]uix*," and they said:

> A service-type like `nginx-service-type` is a declaration. When the manual says it's from `(gnu services web)` it actually means it's a scheme variable that was created in a file called `gnu/services/web.scm`, in the Guix repo. It's much more messy than a package definition though... So you'll have to create the package first, then your service definition will make use of it.

I didn't really understand that answer, completely, but what I did understand is: Yes, writing a package definition is the right place to start.

Before I do that though, I'm going to just try and run it plainly - download the latest binary, config, and run, following the instructions here: `https://writefreely.org/start`

Which... when I try to run the binary inside that, gets me `bash: ./writefreely: No such file or directory`. The file is executable, so this - in the past - has meant it's for 32bit and I'm on 64bit, or vice-versa.

Running `uname -a` I see I'm on `x86_64`, and the binary is for `amd64`. I think those are different platforms, so I guess it's time to build the package! I was going to do that as the next step even if the binary worked.

Writefreely's documentation on this is a bit terse: `https://writefreely.org/docs/latest/developer/setup.md`

I need Go 1.10 or higher, and Node.js

Let's... this sounds like a good opportunity to learn how to do an environment with *Guix*.

But, so I don't get confused later, I'm going to write down the process ahead - I'll probably have to take a break soon, and don't want to lose my stride:

- First, I want to build and run WriteFreely, however I can get it to work, so I can learn how to do that.
  - Learn how to make a Guix environment for tinkering in
  - Learn how to build Writefreely
  - Learn how to configure and run it.
- Then, I want to write a package definition for it.
- Then, I want to define a service for it.
- Then, I want to... run it! I'll be running my own writefreely instance! That's cool!

Okay so uh... I *think* I can just do... `guix environment` to make a little spot to muck around it.

And then `guix install go` and `guix install node` to install those two packages...

Hmm well even if I exit that environment I still have go installed so clearly I did that wrong. But I don't think it really matters.

Writefreely's docs tell me to run this command:

```
go get -d github.com/writeas/writefreely/cmd/writefreely
```

Which holy crap fetches a lot of dependencies. Modern software is crazy complex stacks, huh?

But then... I'm in an empty folder? But now my home directory has a `go/` folder, which has `pkg/` and `src/` subdirectories.

Searching online got me to this: `https://fedi.dev/gytis/build-writefreely-from-source`

Which tells me to go to `~/src/github.com/writeas/writefreely/` and `make build`.

One problem: I don't have `make` installed as a command. I recall reading that the `gnu-build-system` provided such tools, which would probably inform how I accomplish the Writefreely package. But for now? Hmm. I mean, let's try `guix install make`. It's a package, will it be the right one?

Seems to be. I needed to also do `GOPATH=~/go` and `export PATH="$PATH:$GOPATH/bin"`

`make build` gets me an error - apparently I don't have `as`, whatever that is. Poking around online hints me toward installing the `gcc-toolchain` package - I got to this issue whose t itle mimics my error, and then searched the *Guix Reference Manual* for "gcc": `https://github.com/tensorflow/tensorflow/issues/4984` I get a new error now:

```
In file included from /gnu/store/h90vnqw0nwd0hhm1l5dgxsdrigddfmq4-glibc-2.28/include/er
 from cgo-gcc-prolog:29:
/gnu/store/h90vnqw0nwd0hhm1l5dgxsdrigddfmq4-glibc-2.28/include/bits/errno.h:26:26: fata
```

Well. Sounds like maybe installing `glibc` was the wrong package?

But I'm at a loss as to... what package would be the right one. I don't understand enough of the various compilation toolkits to be doing more than taking guesses.

So, let's paste error messages into search engines and see what comes up. This thread: `https://lists.gnu.org/archive/html/bug-guix/2013-02/msg00024.html`

It says to install `linux-libre-headers`. Which tells me to remove `glibc`. I do, and then install `linux-libre-headers` without issue.

Let's try `make build` again? Damn! Same error: `linux/errno.h: No such file or directory`. That thread was the only result for `"guix" "linux/errno.h: No such file or directory"` so now I'm truly at a loss.

Well, it does sill look like it's looking in the store for glibc. Is that perhaps the problem? Let me try garbage collecting, can't hurt. Didn't help, though.

Omg. I'm an idiot. A skim-reading, half-illiterate idiot. So I uninstalled linux-libre-headers and reinstalled glibc and at the end, it says this:

```
export CPATH="/home/emsenn/.guix-profile/include${CPATH:+:}$CPATH"
export LIBRARY_PATH="/home/emsenn/.guix-profile/lib${LIBRARY_PATH:+:}$LIBRARY_PATH"
```

So I did that and then did `make build` and, it worked.

I'm learning: read slower. It's been a big help so far in doing all this, but gosh. It's hard - there's so much I can't understand, it's hard to read every line.

Okay. Well, a lesson to anyone who reads this: I said I was at a loss and kept plugging at it, and got it to work.

Moving on!

I'm using this still: `https://fedi.dev/gytis/build-writefreely-from-source`

It says to run this command: `./cmd/writefreely/writefreely --config`

That works, takes me through the configuration process. Now, `make run` should get me an instance running at the configured address - `http://localhost:8080` for now.

It works! There's no stylesheet - whoops, I missed a step- `make ui`.

Which, of course, errors. I'm going to read this error. Slowly.

So I input `make ui`, and it tells me it's entering the directory `./less/`. It then says shows me the output of the `which` command, telling me there's no `lessc` found. That message repeats several times.

Then it says something I can't interpret so well:

```
node_modules/.bin/lessc app.less --clean-css="--s1 --advanced" ../static/css/write.css
/gnu/store/q19l04vd2za80mk1845pz7r8cz29qk43-bash-minimal-4.4.23/bin/sh: node_modules/.
```

I install `node`, then do `cd less && npm install less less-plugin-clean-css`.

I cd back into the writefreely directory and then do `make ui`. It fails - `lessc` complains it can't find `env`. I do `which env` to see where it is, then edit `lessc` to match.

I `make ui` again, and no errors. I `make run` and go back to `http://localhost:8080` and... it's there and has CSS. Wonderful.

This is what my goal product is.

Except... it's complaining that it can't store cookies. Whoops - I missed a step, between all the hopping between manuals - runnign `writefreely --gen-keys` from within the `./writefreely` source directory.

So. Cool! I have a functional writefreely instance.

Now I have to learn how to do that again, but... as a package.

Planning emsenn.net, Summer 2019

I've been using the domain as my "personal website," at `https://emsenn.net`, in one way or another for a bit over a year now, though what that means has shifted over that period.

I've recently decided to stop sharing so many of my drafts as I work on them, at the advice of @yam655@sonomu.club, who was recently speaking with me about building a community and audience around my writing.

So instead of treating my website as a document repository, I'm going to treat it as a means of communication - more specifically, a way of *broadcasting* long-form text to *subscribers*. In this context, "broadcasting" means "sending, but not receiving, information," and "subscribers" means "people who have chosen to receive the broadcast."

Since I've been using *Mastodon* software service for a while now, I've grown to like the *Fediverse*, the rough conglomeration of cultures that communicate using the *ActivityPub* protocol.

So it seemed natural to move to *WriteFreely*, a *federated* blogging platform. The software, *open-source* of course, is maintained by the same people who run the *Write.as* web service. There, one can post anonymously or run their own blog.

That's what I'm choosing to do, for now - I've gone ahead and signed up for their "Pro" plan, which has let me set up a custom domain.

So now, if you go to `https://emsenn.net`, you see a title (my name), a list of pinned posts, and then a chronological feed of my posts - most recent first. At the bottom of the page, there's a field where you can put your *email* address to *subscribe* to new posts - they'll show up in full in your email. If *RSS* feeds are your preference, they're available at https://emsenn.net/feed/.

There's a web interface for adding new posts and managing the site, but I'll be adding posts directly from *Emacs*, my text-editor of choice. In it, I write files in the *Org-mode* format, and use the *writefreely.el* script to upload them to *Write.as*.

...Actually in playing with it, I can't get that to work quite properly if I try and update a post. So what I'll probably do instead, for now, is export to *Github-Flavored Markdown* with *Ox-GFM* and then use the web interface to post and update. A bit clunky, but as I'm no longer prioritizing my website as the main place I host my writing, I'm okay with it.

I glossed over it at the start, but that is the main reason I'm comfortable moving my website from an ultra-light just-HTML thing to a web service: I plan on offering that sort of HTML document through some sort of store-front, for lack of a better term. I'm already doing something like that with my writing with Teraum - you can see an example at https://emsenn.itch.io/crown-of-idbol-bark/ - though at the moment there's only a PDF available.

With that burden lifted, to be placed somewhere else in the near future, `emsenn.net` can be a broadcast platform, and *WriteFreely* seemed like an

adequate choice to get that done.

Planning a Personal Media Storefront Implemented in Hugo

Recently I decided that I was going to make a change in how I presented my writing online, away from a traditional website and toward a "storefront." I was pushed toward the decision when looking at how I wanted to present my writing related to Teraum, a fantasy setting I'm implementing as some stories and other texts.

While I have HTML files for each document, I'm not making them into a website. Instead, I've listed them as "games" on itch.io, a web service for hosting and selling independent video (and other) games.

That's not too different from how my website used to be, before I switched it to running *WriteFreely*.

I was using *Hugo* and *Ox-Hugo* to turn *Org-mode* source files into *HTML*, and I'll probably roughly follow that same toolchain.

So let's think about what the site would have: There'd be an index page that would explain the site and probably present the most recent additions to the "store".

There'd be a page for each "item" in the store, which would list info about it, its different formats in the latest version, and past versions.

Which implies the existence of a directory that has all that. I imagine it'd probably be best to do it `/store/{slug}/{version}/{slug}.{format}`.

So the *HTML* rendering of version `1.0.0` of "A Shaggy Dog Tale" would go in `/store/shaggy-dog-tale/1.0.0/shaggy-dog-tale.html`.

The store page itself - the one that people would go to, to find their download link...

Actually, I should be able to link to an arbitrary source for the document - so while I might store things in that place I said, I'll want to accommodate them potentially being somewhere else.

When I was thinking about this last night, I realized that in a way, I'm building a sort of package management system, for my own writing.

Hugo probably makes that easy, because it lets you use *TOML* data files to supplement page contents.

Sorry if this feels fragmented to read - notes like this are written primarily for my benefit, and this is a bit project, conceptually.

I'll back up. There has to be a homepage. Then a page for each document that's available through the storefront.

It's been a few months since I've used Hugo, but as I recall, it encourages you to organize your posts under directories based on *content type*, which

helps it decide what *layout* to use.

I think I can comfortably use these *content types*:

- `documents` for mostly-text things, like books, essays, collections of letters, and so on.
- `images` for graphic assets, illustrations, scans of paintings, so on.
- `audio` for music and other sounds.
- `video` for video assets.

So, going back to the example, there'd be a page at `/documents/shaggy-dog-tale/`.

Oh and to be clear, I'm picturing all this living at `https://store.emsenn.net`.

Each page would have a bit of text about the document - with how I currently do things, I could copy any given document's "Introduction" section, since it is an out-of-context explanation of the document to follow, plus its license and so on.

It'd also have links to each available format of the current version, then a summary of past versions, and links to those downloads.

That... is a little trickier to build. I'm picturing that each page would be

So... each page - each *Markdown* source file - would basically be a package definition? Okay - so I'm writing a package definition schema implemented as a *Hugo*-compatible *Markdown* file, and layouts that render those.

That seems pretty comprehensible. I'm probably reinventing the wheel in some way - I mean, I could just release packages for another package manager and use whatever their web interface is.

But, here, I'll be able to really control how every store page looks, and I'm not sure but there might be features I want for a document-focused package store that existing software-focused package managers have. And beyond that, I'm not looking to build a full package manager, but just a static listing and an interface for accessing it.

The next step will be to create a basic *Hugo* site to build this out on, and then start to implement the media package definition schema and layouts for rendering it. That sounds complex, but unless I've forgotten more about *Hugo* than I think, it should be fairly easy at first. The difficulty will be in extending it as I use it and discovere more use-cases.

Defining a Media Package Definition and Channel

Earlier today I wrote about creating a "personal media storefront," a place where I could maintain a listing of all the things I've made, where they're

available, and in what formats.

I spent some time prototyping out a collection of *Hugo* templates for that purpose, and realized that basically what I'm building is a package reposistory, but for media, not software. Since then I've tried to think of things in that way - tricky, as I don't know that much about how software package managers work. Luckily, I've been learning to use *Guix* recently, and that gave me an exposure to its package definitions, and gave me some vocabulary to use moving forward.

What I was calling a "storefront" - a bad metaphor since I had no intention of charging for access to content - I'll now call a "channel." I'll have my channel, and you, another content creator, would have yours.

I'm picturing a shell script based on `curl` that can do something like `mepach download --channel "https://mepach.emsenn.net/" shaggy-dog-tale`.

This would `GET` a `json` file at, say `https://mepach.emsenn.net/get/shaggy-dog-tale`, which would have a package definition.

"Mepach" here is a potential name for this project, short for MEdia PAckage CHannel. I've just searched online and it doesn't seem to be an existing term in computing.

At `https://mepach.emsenn.net/shaggy-dog-tale` there would be an HTML representation of the project, and that's what I've been working on most recently.

A thought - in my previous devlog and prototyping, I used different *Hugo content types* to categorize content. I don't think I should do that, so it is easier to make assumptions about a package's location.

Here's the example package definition I made earlier:

```
+++
title = "A Shaggy Dog Tale"
description = """This is a story that doesn't go anwhere."""

[v."1.0.1"."github"]
location = "https://github.com/emsenn/master/"
formats = ["org", "html"]
[v."1.0.1"."local"]
location = "/store/"
formats = ["org", "html"]

[v."1.0.0"."local"]
location = "/store/"
formats = ["org"]
```

```
+++

This story is about a dog that does a lot of things, but never
 really gets anything done.
```

I've split it into two sections because it uses two languages - *TOML* for the *Hugo front matter* and *Markdown* for the content.

The first parameters are general facts about the document the package is representing - for now, its `title` and a `description`. Neither needs to be required; a working title can be inferred from the package definition's file name, which is required to be the package's "slug," or identifier. The example here is saved at `./content/shaggy-dog-tale.md`.

The `description` is wrapped in three `"` instead of one so it can use linebreaks - though I might be doing that wrong; I'm not familiar with the finer points of *TOML* yet.

Below those general parameters is information about different versions.

Each *dictionary* representing a version of the document needs to start with `v`. The dot in-between separates that from the version number itself, and the version origin - more on that shortly. So that the version number's dots don't get parsed, it's wrapped inside quotes. The origin is also wrapped in quotes, since it might contain special characters.

The origin is where the document can be found - whether it's available locally, or through a *Git forge*, or some other website or protocol.

In the example above, version `1.0.0` is available through the `local` origin, and version `1.0.1` is available through the `local` and `github` origin.

Each *dictionary*, then, specifies a version and origin. I don't have a term for this dictionary, perhaps "source dictionary"?

For now, I'll go with that. So each source dictionary contains two keys, the `location` and the `formats`.

The `location` says where the files are. The `formats` say which formats are available at that location.

I should be clear, this is the internal format that's used to generate the actual package formats - I need to develop a better grasp of terms here so I can be more clear. This *TOML* + *Markdown* package definition is used on the backend, and will be turned into an actual useful package definition by the so-called media package channel generator I'm working on.

I've already built a badly-executed *Hugo layout* for turning this into an *HTML* page, here's an example of how the above looks. **Links will be broken.**

A Shaggy Dog Tale

- **1.0.1**:
  - github:
    * html
    * org
  - local:
    * html
    * org
- **1.0.0**
  - org

So to spell it out:

- I'll write a package "definition" in *TOML* and *Markdown*. I don't like calling this the definition, but don't know what else to call it.
- Then, *Hugo* will turn that into an *HTML* and a *JSON* file.
- Those get made available somewhere on the Web.
- Someone can use them as a package definition to download their version and format of the document of choice.

The main use here is still, I think, that I'll have a website where I'll be able to look up any mirror and old version of my documents.

But the packaging stuff is cool to look at, too.

Looking at my Org-mode Files

I've been using *Org-mode* more and more heavily for the past... year, or so.

In that time, I've shuffled a lot of files around in my org-mode directory, but without always making sure older parts continued to make sense. Most importantly, I have a lot of little *Org-mode macros* scattered across different projects, which means every time I export anything from that project, it has to load all the macros, and that takes time.

So, as I said, I have a a directory named `org/` in my home directory.

That directory in turn contains these directories:

- `clients/`

- `personal/`
- `projects/`
- `triage/`

The `clients/` directory contains a directory for each of my current clients, as well as a directory called `archive/` that contains a directory for each of my past clients. Client directories are similar in layout to a project directory.

The `personal/` directory is also laid out like a project, which is, it contains two subdirectories, `pub/` and `src/`. `src/` is where the *Org-mode* files themselves are located, and `pub/` is where any exported reports go, such as logs I've rendered exported to PDF to print out.

The `projects/` directory has most of my stuff - and when I say that, I mean it; almost everything I make is one or more *Org-mode* files. Explaining why is outside the scope of this post.

Each project has a `src/` and `pub/` directory. Here is a list of other files and directories my projects might have:

- `src/setup.inc/` which would contain project-wide *Org-mode* configurations. It would be used by other source files in that project.
- `src/index.org/` which would be the project's "homepage," if the project is rendered as any sort of collection of, or individual, HTML documents - which it almost certainly will be.

I'm honestly not able to think of anything else that is common between different projects.

Finally, the `triage/` directory contains *Org-mode* files that I haven't decided a better place for yet - such as a new piece of writing that I'm unsure if it'll be a letter, post, or what. For example, the file I'm writing this text in is in `triage/looking-at-my-org-mode-files.org`.

Having written most of it, though, I'll probably move it into `projects/posts/src/looking-at-my-o`

Anyway, that's the layout of my Org-mode directory, as it currently stands.

Looking at How I Organize a Document

Overview

In this post I'm going to explain how I'm organizing my documents these days. By documents, I mean written texts - stories, essays, posts like this

one, but also, software. Software might not seem like a document, but I use the *literate programming* paradigm when I work with a computer, so I write my software as snippets of code inside longer essays.

I'll use my *Brutstrap* project as an example in this post. It's a relatively simple *CSS* theme I made, so should be an opportunity to demonstrate a lot of things without its content getting in the way.

Document Project Layout

Each document exists in its own directory on my computer, under the `~/org/projects/` directory. It's called `projects` because within *Org-mode*, the name for collections of files published together is a "project," so each document exists as a "document project." Brutstrap's "identifier" is `brutstrap-css` - that means it's at `~/org/projects/brutstrap-css/`.

The root of that directory contains the document in various formats (*HTML*, plain-text, *Markdown*, and others), as well as a `README.md` and a `media-package.json`.

There's also a subdirectory, `src/`, which contains an `index.org` and subsubdirectory, `build/` which has in it an `index.html` and `brutstrap.css`.

The only file I edit by hand is `src/index.org`, which is an *Org-mode* text file. All the other files are created by using *Org-mode* features on that file.

The Source File

Here's a walkthrough of what's in that file. I'll break it up into the frontmatter, introduction, document contents, and supplements.

Front Matter

```
#+TITLE: Brutstrap
#+AUTHOR: emsenn
#+EXPORT_FILE_NAME: ../brutstrap
#+MACRO: document-type CSS theme
#+HTML_HEAD: <link rel="stylesheet" type="text/css" href="./src/build/brutstrap.css">
{{{document-brief}}}
#+TOC: headlines local 4
```

Let's look at this line by line:

- `#+TITLE: Brutstrap`

  - This sets the document's title. Fairly self-evident. It's used when exporting the document, and also available as a *macro*: `{{{title}}}`. I use *Org-mode macros* a fair amount, they're a way to quickly insert pre-assigned text.

- `#+AUTHOR: emsenn`
  - This set's the document's author. Works similar to title.

- `#+EXPORT_FILE_NAME: ../brutstrap`
  - This tells *Org-mode* where to put exported copies of the file. When I tell *Org-mode* to export `./src/index.org` as *HTML*, it'll export to `./brutstrap.html`.

- `#+MACRO: document-type CSS theme`
  - This sets the `document-type` macro, which is used by some other macros to say what sort of document this is - in this example, it's a "CSS theme". Due to how it's used, the document-type is assumed to be a singular noun.

- `#+HTML_HEAD: <link />`
  - I cut out the specifics, but this line tells *Org-mode*: when exporting as an HTML file, link to `./src/build/brutstrap.css` as a stylesheet.

- `{{{document-brief}}}`
  - This is a macro I've defined in my *Org-mode* configuration, so it's available in all my *Org-mode* files. Here's what it's defined as:

    `"{{{title}}}" is a {{{document-type}}} created by {{{author}}}.`

    As you can see, it makes use of the macros defined above.

- `#+TOC: headlines local 4`
  - This generates a table of contents linking to sections and through three depths of subsections. (1+3=4)

Past here, the document is divided into three sections: an "Introduction," the document contents itself (here, "Brutstrap,") and "Supplements."

Introduction Section Here's the Introduction:

```
* Introduction
  :PROPERTIES:
  :CUSTOM_ID: introduction
  :END:
{{{document-is-draft}}}

In this document I explain...

{{{document-eli}}}
```

Again, line by line:

- `* Introduction` declares the start of what *Org-mode* calls a *subtree*, but what normal people would call a "section". How many asterisks lead determine the depth of the section - `**** Pumpkin` means that "Pumpkin" is a subsubsubsection.
- `:PROPERTIES:` declares the start of what *Org-mode* calls a *drawer*, which is container for data associated with an element of the document - in this case, the Introduction section.
- `:CUSTOM_ID:` declares what the identifier for this section should be, when it's exported. It's a part of the `PROPERTIES` drawer that was just opened. This is the name that anchor links in an *HTML* export will point to, for example. If this isn't set, *Org-mode* will generate an identifier automatically, something like `org3507c8b`.
- `:END:` declares the close of the `PROPERTIES` drawer.
- `{{{document-is-draft}}}` is another macro from my *Org-mode* configuration:

  ```
  This {{{document-type}}} is a /draft/: it is unfinished and that
  should be considered when reading it. Special attention should be paid
  to text marked DRAFT, EDIT, or TK.
  ```

- `In this document I explain...` is an abridged version of the paragraph that's actually there, that explains the broad strokes of what *Brutstrap* is, how it's meant to look, and so on.

- =This post was created by me, emsenn, in the United States. To the extent possible under law, I have waived all copyright and related or neighboring rights to this document. This document was created for the benefit of the public. If you're viewing it on a remote server, you're encouraged to download your own copy. To learn how you can support me, see https://emsenn.net/donate. is yet another macro from my *Org-mode* configuration. This one is a bit unusual, it's set to `{{{document-eli-$1}}}`. *Org-mode* passes text after the macro name, between paratheses, as arguments, separated by commas. So, `{{{document-eli(default)}}}` translates to `{{{document-eli-default}}}`. Why don't I just do `{{{document-eli-default}}}` in the source file then? This way I can change how macro is rendered across all files, just by updating the `document-eli` macro. So, what does `document-eli-default` expand out to?

  ```
  This {{{document-type}}} was created by me, emsenn, in the United
  States. To the extent possible under law, I have waived all copyright
  and related or neighboring rights to this document. This document was
  created for the benefit of the public. If you're viewing it on a
  remote server, you're encouraged to download your own copy. To learn
  how you can support me, see
  [[https://emsenn.net/donate][https://emsenn.net/donate]].
  ```

  Oh - it might help to say what "eli" is short for: *editorial and licensing information*: a declaration of the author and the rights assocated with the document. I assume I'm the author, and that the rights are "released into the public domain."

Document Contents That's it for the "Introduction". The "Brutstrap" section contains the *Brutstrap* theme itself - explaining how I *literately program* CSS is beyond the scope of this document. In short, using *Babel*, I'm able to write short blocks of CSS, and extract them out to `./src/build/brutstrap.css`.

Supplements Section The "Supplements" section contains four subsections:

- "Example Texts" This section contains two subsections:
  - "Complete Stylesheet," uses *Org-mode's* inclusion feature to pull in `./src/build/brustrap.css` - which is itself extracted from the "Brutstrap" section above.

  - "Demonstration HTML," is a source block that, again using *Babel*, is written to `./src/build/index.html`

- "Index"

  - This section uses *Org-mode's* dictionary list feature to present a list of proper terms mentioned in the document. Here's an excerpt from *Brutstrap's* index:

    ```
    - brutalism :: {{{ref(brutalism)}}}
    - Brutstrap :: {{{ref(brutstrap)}}}
    - CSS :: {{{ref(css)}}}
    - HTML :: {{{ref(html)}}}
    ```

    This introduces a new macro: `ref`. This is, like the others, defined in my *Org-mode* configuration, and is: `{{{define($1)}}}` `{{{source($1)}}}`. `define` is defined as `{{{$1-definition}}}` and `source` is defined as `{{{$1-source}}}`.. For each term, then, I have two macros, for example for Brutalism I have `brutalism-definition` and `brutalism-source`. The definition is simple a sentence defining the term, and the source is... a call to yet another macro: `{{{source-link(https://en.wikipedia.org/wiki/Brutalist_architecture)}}}`. `source-link` is defined as `See [[$1][$1]]` which is *Org-mode* formatting for a link whose anchor text is the URI it directs to.
    So, the final entry for *brutalism* would be:

    ```
    - Brutalism :: A style exaggerating the pragmatic functionality
           present in modernist design. See
           https://en.wikipedia.org/wiki/Brutalist_architecture.
    ```

- "Media Package"

This section contains a *JSON* source block which implements a specification I'm developing for what I'm calling a *media package*, which is a manifest of information about a piece of media content.

Explaining that is beyond the scope of this document, in part because it's in development, so I'll just show the *JSON* here:

```
"brutstrap-css": {
    name: "Brutstrap",
    creator: "emsenn",
    license: "CC0",
    languages: ["en_US","CSS","HTML"],
```

```
    versions: {
"current": {
    "sourcehut": {
"location": "https://git.sr.ht/~emsenn/brutstrap-css/blob/master/",
"formats": ["html","md","pdf","tex","txt"]
    }
}
    }
}
```

- "README"
  - The "README" section is used to generate a `README.md` at the root of the project, as that file is often used by *Git forges* to generate the project's "homepage" on the forge. It contains three thing:
    * It sets the `PROPERTY` of `EXPORT_FILE_NAME` to

    `../README`, so that when this section is exported, as say *Markdown*, it will be exported to `./README.md`.
    * Then, it has the line `*** {{{title}}}` which creates a subsection under itself using the document's title.
    * Then, it includes the "Introduction" section from `./src/index.org` - itself - but only the contents, not the title.

  Here's what the section looks like in the source file:

  ```
  *** README
      :PROPERTIES:
      :EXPORT_FILE_NAME: ../README
      :CUSTOM_ID: readme
      :END:
  **** {{{title}}}
  #+INCLUDE: "./index.org::#introduction" :only-contents t
  ```

  And here's what `./README.md` looks like - with the paragraphs cut down.

  ```
  # Brutstrap

  This CSS theme is a *draft*...
  ```

```
In this document I explain...

This CSS theme was created by...
```

Conclusion

And so that's it, how I manage a document as a project, exporting it to various different formats, maintaining a README, and so on, by just editing one file for each document, one at `./src/index.org`.

Index

brutalism A style exaggerating the pragmatic functionality present in modernist design. See https://en.wikipedia.org/wiki/Brutalist_architecture

Brutstrap

CSS

JSON A human-readable data format. See https://json.org/.

literate programming

Markdown A way of adding style notation to plain text. `https://daringfireball.net/projects/markdown/`

Org-mode Babel A software tool for using code integrated into *Org-mode* documents. See https://orgmode.org/worg/org-contrib/babel/intro.html

Org-mode macros A way to replace text in a source document when it is exported to a rendered document. See https://orgmode.org/manual/Macro-replacement.html

Org-mode A way of formatting plain data inside text documents. See https://orgmode.org/ or #OrgMode.

Planning a Personal Website with Hugo

So I've been going through my various online representations, trying to figure out what each one is for, and what I'm missing. Part of that has been moving my "blog," such as it was, off of my main homepage and moving it to a dedicated platform for *broadcasting* writing. Now, I'm using *WriteFreely* at

`posts.emsenn.net` to share some of my longer but more informal writing - this text is probably going to go there.

This leaves `emsenn.net` free to be mine, rather than belonging to my writing. You might think this would leave me without much content to put there, and in way, that's true, but there's also still a *lot* of information that one can provide about oneself to be a filled-out digital representation.

A note - I'm going to be using terms like agent, representation, avatar, user, person, and so on as I write about this stuff, and I'll be doing so without what I hope is my usual precision. I'm not sure how I want to train myself to think about these things yet, so I'm not sure what terms I want to use for which concepts. I apologize if the current indecision makes this writing less intelligible or more verbose.

So, `emsenn.net` is free to be my "personal website." Now... to think about what that means. It'd need to have information about me, as a person. I often find myself wanting to link to info about how to donate to me. I maintain a list of things I recommend folk read. I also maintain a list of my recent activities, my plans, and I'd like to start keeping a list of things I'm looking for help with.

There's also things that are a part of the *IndieWeb* that I'd like to add, and other things like a `/now/` page, which would have my most recent activities.

I put some thought into this the other night and I think the way I'd like to implement this is as a static *HTML* website - there's nothing I do so rapidly that people need a second-by-second update of it.

Hugo is my static site generator of choice - it's powerful enough to do everything I've wanted to do with it, without relying on your final site having *Javascript* (or being very particular about avoiding it.)

I want to do things properly this time though - and by properly, I mean in such a way that what I'm making can be useful for other people who'd like to make a site like mine, but with their own information.

As I understand it, this means organizing the components as *Hugo themes*. *Hugo* uses what it calls *layouts* to determine how to render a document, and follows a *lookup order* to determine which layout to use.

So, I think I'll start by making a new project, `hugo-personal-website-theme`, that'll be a *Hugo* theme, and used by the `emsenn-website` project, that'll be a *Hugo* website.

That's about all I can write on the topic until I take the first steps toward implementation.

Index

broadcast To send a piece of information to *subscribers* or into a public area.

HTML A way of adding layout, formatting, and media to text documents. See https://en.wikipedia.org/wiki/HTML.

Hugo, layout The framework which determines what is in a rendered document and how it appears. See https://gohugo.io/templates/lookup-order/

Hugo, lookup order The order in which *Hugo* looks up which layout(s) to apply to a page. See https://gohugo.io/templates/lookup-order/#readout.

Hugo A software tool for creating rendered documents from formatted text files. See https://gohugo.io/

Hugo, themes Collections of layouts for use by the *Hugo* static site generator. `https://gohugo.io/themes/`

IndieWeb A set of data specifications and software utilities for coordinating the serialization of social data. See https://indieweb.org/.

Javascript An interpretted programming language used frequently in Web development. See https://en.wikipedia.org/wiki/JavaScript.

WriteFreely A software service for broadcasting long text documents. See See https://writefreely.org/ or #WriteFreely.

Implementing a Personal Website with Hugo, Netlify, Org-mode, and Ox-Hugo

I've been reworking my online presence recently, and part of that has been to change how I handle the website at `emsenn.net`.

I've put a fair bit of thought into the generalities of how I want to handle it, but not the specifics. But first, let's look at the parts of my online identity I have set up right now, that I don't have any plans to change:

- I conduct *email* through emsenn@emsenn.net.
- I post publicly at https://posts.emsenn.net. Those posts are also available through an *email list*, at https://posts.emsenn.net/#subscribe. They're also available as an *RSS* feed at https://posts.emsenn.net/feed/ and through the *ActivityPub* protocol at `@emsenn@posts.emsenn.net`.

- I do two-way communication through *ActivityPub* at `@emsenn@tenforward.social`.
  You can view the public posts associated with that account at https://tenforward.social/@emsenn.
- My primary *Git forge* is *Sourcehut*, at https://git.sr.ht/~emsenn. I like its approach to using existing tools properly, rather than adding on new tools, when it comes to stuff like bug-tracking. (Even though I don't use those features.
  - I use *Gitlab* to mirror my projects on *Sourcehut*, at https://gitlab.com/emsenn.
  - I also mirror them on *Github* at https://github.com/emsenn.

I'm not sure what the full scope of `emsenn.net` will be, but I don't plan on it being a *dynamic website* - nothing I do happens so rapidly that I need to run something like that: I can serve up *static* resources like HTML and JSON files, that are compiled well in advance of the request.

I like *Hugo* as my *static site generator* - it's powerful enough to meet my needs without being so powerful I get bogged down in different ways to do things.

So, I know that's one part of my *software stack*: the layers of software used to accomplish a task.

I also like *Netlify* as a *webhost* for *static websites* - they have an incredible amount of features, that don't get in the way until you want them there. That's one end of the *stack*: publishing on *Netlify*.

Netlify can source its sites from three *Git forges*: *Gitlab*, *Github*, and *Bitbucket*. I prefer *Gitlab* to *Github*, since *Gitlab* is funded by users rather than owned by *Microsoft*, so I'll use *Gitlab* as the source for *Netlify*. And since the *Gitlab* repo is simply a mirror of the *Sourcehut Git forge*, which is in turn just a presentation of my local *Git repository*, that gives me four sequenced tools in the *stack*: *Git* to *Sourcehut* to *Gitlab* to *Netlify*.

That still leaves what is in the *Git repo* unsorted, as well as how it gets built by *Hugo*. It's been a bit since I've worked with *Netlify* and *Hugo* and all this, and I was never that experienced with it when I was, so at this point, I'll need to start setting things up before fleshing out that part of the *stack*.

So let's look at instead what I'm working with as the *source code* for the website. It's organized like any of my other projects, under the `~/org/projects/` directory on my server. Here's what the `emsenn-website` directory under that looks like:

```
./
 | README.md
 | emsenn-website.html
```

```
| emsenn-website.org
| emsenn-website.pdf
| emsenn-website.tex
| src/
    | emsenn-website.org
    | build/
   | content/
   |        | <site pages go here>
   | data/
   |     | <site data goes here>
   | themes/
   |       | hugo-personal-website-theme/
   | config.toml
```

The only file I edit by-hand is `./src/emsenn-website.org` - the other files are *tangled* from it - extracted from code blocks written into the text of `./src/emsenn-website.org`.

Here's an overview of that document's sections:

- "Operations" has shell scripts and instructions for working with the source file or otherwise working with the personal site. Any of the shell scripts can be run from inside *Org-mode*, and the results included in any exported document, if that's useful.
- "Hugo Configuration" has *Hugo configuration*. Pretty simple. It's extracted from this source file by using *Babel* to *tangle* (extract) the configuration.
- "Content Pages" has sections for the website's pages. They're exported with *Ox-Hugo*, an *Emacs* package for exporting *Org-mode* to *Hugo*-compatible *Markdown*. These files end up in `./src/build/content/`.
- "Site Data" has some *TOML* files that have information about my recent activities and plans, for now. These files end up in `./src/build/data/`

The themes directory contains a directory, `hugo-personal-website-theme`, which is a *Hugo theme* that I'm working on - I'll probably write about it elsewhere.

Index

Markdown

Hugo

Hugo theme

TOML

Ox-Hugo

Org-mode

Hugo configuration

Babel

Babel tangle

ActivityPub

Bitbucket

email

email list

Git

Git repository

Git forge

Github

Gitlab

Hugo

Microsoft

Netlify

Org-mode

Ox-Hugo

RSS

software stack

Sourcehut

website, dynamic

website, static